

G S o C 2 0 2 6

OpenVINO Optimization for VLA Model Inference

Progress update · first two weeks

Mohamed · unifolm-vla · OpenVINO runtime

What I worked on

From understanding the project architecture to validating the first export strategy.

Three focus areas across the two weeks: runtime onboarding, model tracing, and turning the export blocker analysis into concrete experiments.

01 OpenVINO runtime onboarding

02 unifolm-vla architecture tracing

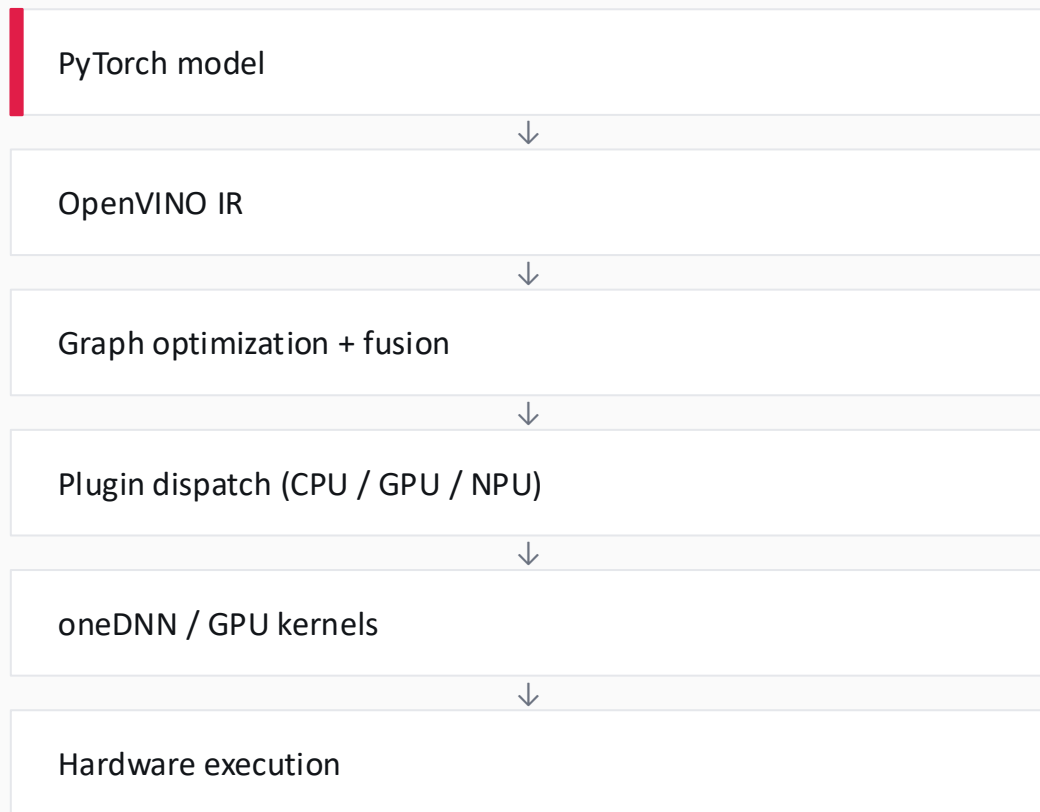
03 Export blocker reproduction

04 Single-step DiT export

05 IR / runtime validation

06 External denoising loop prototype

OpenVINO as a compiler + runtime stack



THE GOAL

**Convert iterative VLA inference
into efficient hardware execution.**

Why it matters for this project

VLA action generation is not a single forward pass. It is repeated denoising, closer to ImageGenerationPipeline than to standard autoregressive LLM inference.

Diffusion vs VLA: structural equivalence

D I F F U S I O N

Latent denoising

Scheduler timesteps

Repeated UNet / DiT

Iterative image refinement

V L A

Action denoising

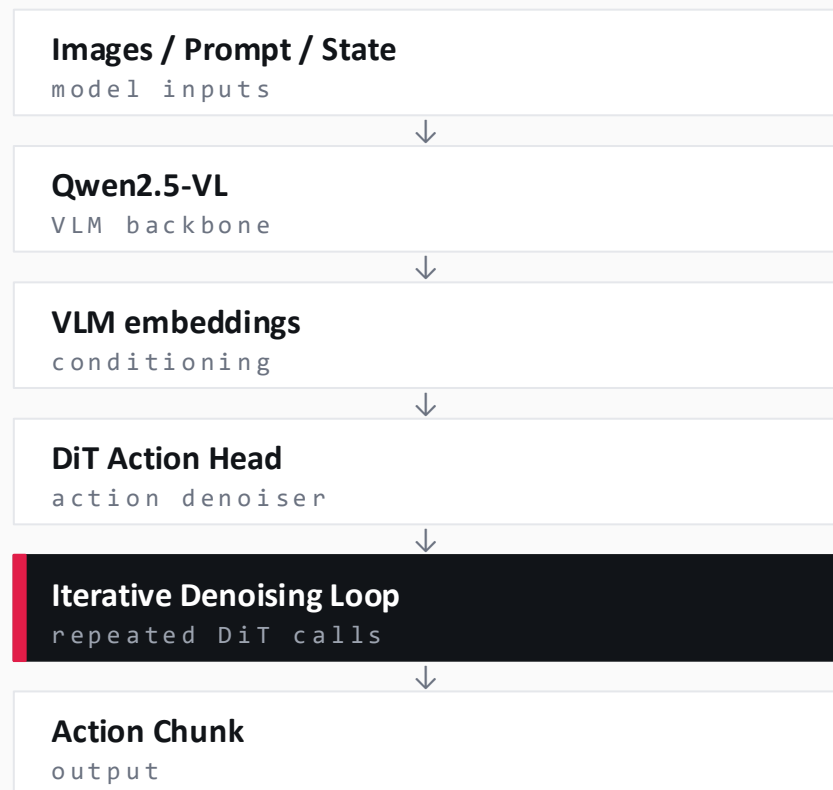
Action timesteps

Repeated action DiT

Iterative trajectory refinement

VLA inference structurally resembles diffusion-style iterative generation.

unifolm-vla inference path



THE BOTTLENECK

DiT_ActionHeader.py · line 251

```
1 # DiT_ActionHeader.py (Line 251)
2 for t in range(num_steps):
3     # ... DiT execution ...
4     actions = actions + dt * pred_velocity
```

Bottleneck hypotheses

- Repeated DiT execution per timestep
- Repeated cross-attention compute
- AdaLayerNorm modulation overhead
- Memory movement + kernel dispatch

AdaLayerNorm sits inside the repeated path

The modulation pattern

```
timestep_emb → SiLU → Linear  
              → split(scale, shift)
```

```
LayerNorm(x) * (1 + scale) + shift
```

Appears in both transformer block and final output modulation.

WHY THIS MATTERS

DiT runs once per denoising timestep.

If decomposed into separate LayerNorm, multiply, and add operations, that overhead repeats across every action denoising step.

Result

The fusion target from the proposal sits directly on the critical execution path. A concrete place to look once profiling begins.

Four blockers, all about graph boundaries

BLOCKER	PROBLEM	FIX DIRECTION
Python loop DiT_ActionHeader.py : 251	Graph explosion / unrolling under tracing	External scheduler loop
torch.randn DiT_ActionHeader.py : 239	Stochastic ops baked into graph	External noise input
torch.autocast unifolm_vla.py	Runtime precision contexts inside forward	Compile / runtime precision
BatchFeature DiT_ActionHeader.py : 186	HF dict input, non-clean graph boundary	Tensor-only wrapper

Export problem is about graph boundaries and orchestration, not the transformer itself.

Move the loop out, compile one step

BEFORE

Export full denoising loop



Huge static graph



Unrolled control flow, opaque kernel mix

AFTER

External Scheduler Loop

+

Compiled Single-Step DiT



Reusable runtime graph

THE NEW GRAPH BOUNDARY

```
1 # export_tests/single_step_dit_wrapper.py
2 class SingleStepDiTWrapper(nn.Module):
3     def forward(self, vl_embs, actions, state, timesteps_tensor):
4         # ... executes exactly one DiT pass ...
5         return pred_velocity
```

Single-step DiT exported and validated

IR ARTIFACTS GENERATED

```
artifacts/opencvino_ir/  
├── single_step_dit.xml    713 KB  
└── single_step_dit.bin   1.1 GB
```

TERMINAL OUTPUT

```
[INFO] Converting model to OpenVINO IR...  
[INFO] Saving IR...  
[SUCCESS] Single-step DiT converted and saved.
```

VALIDATION CHECKLIST

- ✓ IR loaded successfully
- ✓ OpenVINO compilation succeeded
- ✓ Runtime inference succeeded
- ✓ Output shapes match exactly (1, 8, 7)

***The core transformer graph is exportable.
The surrounding orchestration was the real problem.***

External loop calling the compiled step

Python Scheduler Loop

host orchestration



Repeated OpenVINO Single-Step Calls

compiled DiT runtime



Action Refinement

Euler integration

THE ORCHESTRATION

```
1 # export_tests/opencvino_external_denoising_loop.py
2 for t in range(num_steps):
3     ov_results = compiled_model(ov_inputs)
4     pred_velocity = list(ov_results.values())[0]
5
6     # Euler integration refinement
7     actions = actions + dt * pred_velocity
```



External orchestration
works

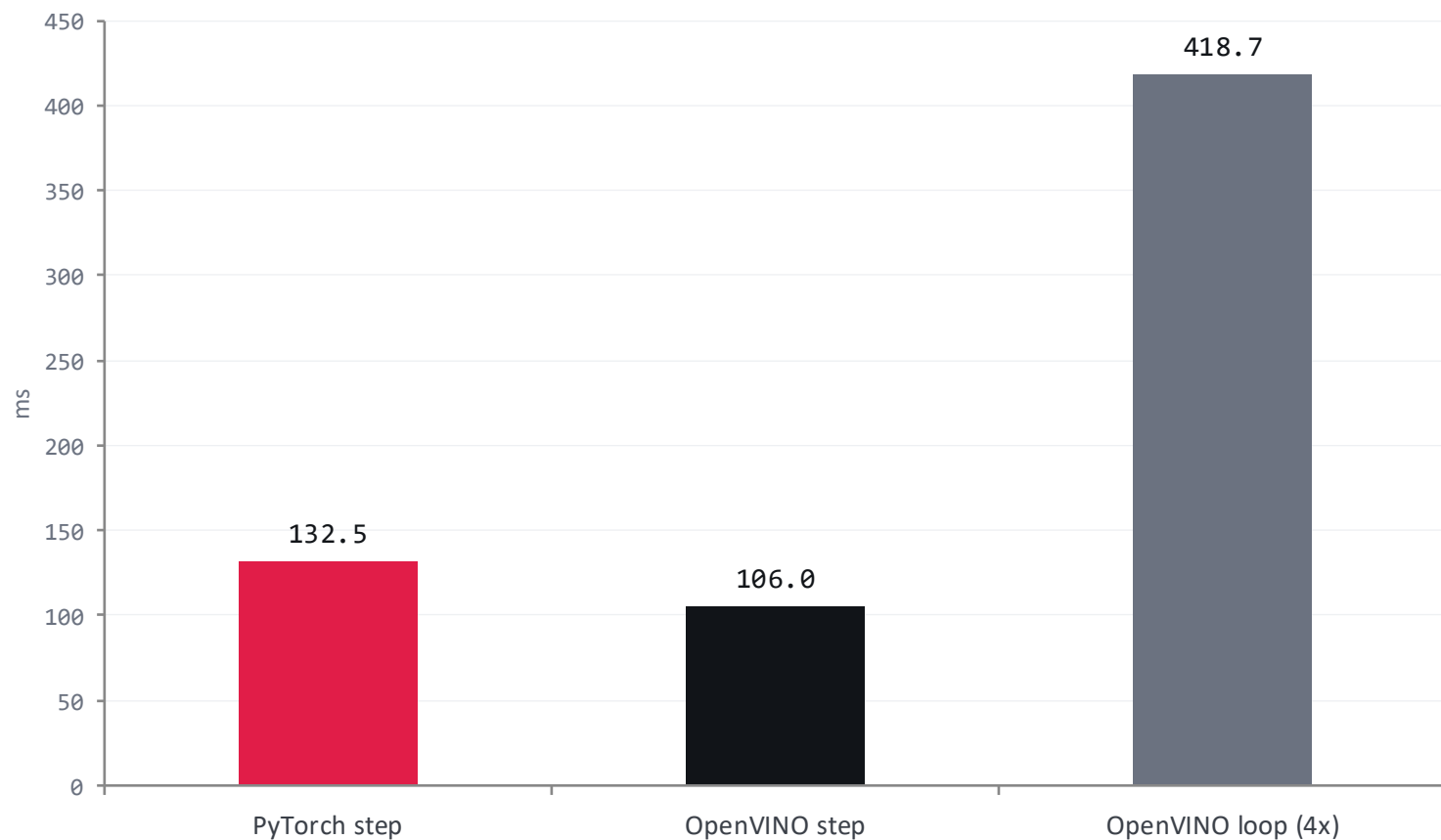


Repeated OV execution
works



Action updates preserved

Baseline latency on local CPU (Apple M4)



OBSERVATIONS

~20%

OpenVINO single-step
advantage over PyTorch

Loop overhead is linear.
No orchestration penalty.

Benchmark framework is built.
Ready to rerun on Intel cloud
when access opens.

Where we are now

WHAT THIS PROVES

- 01

Model structure matches the proposal

VLM backbone + repeated DiT denoising.
- 02

Runtime cost lives in the repeated DiT path

Not in the VLM wrapper.
- 03

AdaLayerNorm sits on the critical path

Meaningful fusion target.
- 04

Single-step DiT is exportable to IR

Validated end-to-end.
- 05

External scheduler strategy is viable

Compiled step + host loop.

CURRENT LIMITATIONS

- Intel cloud access pending

Documented; continued with local export and validation in parallel.
- Results are CPU-only

Treating numbers as correctness signals, not final performance.
- External loop still a prototype

Needs input naming, dtype handling, and parity checks.

From validation to baseline performance.

- | | | |
|----|--------------------------------|--|
| 01 | Numerical parity | PyTorch vs OpenVINO output for the single-step wrapper. |
| 02 | Clean the external loop | Make it a reusable baseline implementation. |
| 03 | Intel hardware | Resolve cloud access; rerun export and benchmark scripts. |
| 04 | Inspect generated IR | How AdaLayerNorm and attention represent after conversion. |
| 05 | Begin profiling | VTune / OpenVINO profiling on the repeated DiT step. |
-